receives an Access Token granting it access to downstream services on behalf of that user.

In this section and the next one, you'll see how to secure Catalog Service and Order Service by requiring a valid Access Token to access their protected endpoints. In the OAuth2 authorization framework, they play the role of OAuth2 Resource Servers: the applications hosting protected data that a user can access through a third party (Edge Service, in our example).

OAuth2 Resource Servers don't deal with user authentication. They receive an Access Token in the `Authorization` header of each HTTP request. Then they verify the signature and authorize the request according to the content of the token. We have already configured Edge Service to send over an Access Token when routing a request downstream. Now you'll see how to use that token on the receiver side. This section will guide you through securing Catalog Service, built on the imperative Spring stack. The next section will show you how to achieve the same result in Order Service, built on the reactive Spring stack.

12.2.1 Securing Spring Boot as an OAuth2 Resource Server

The first step in securing a Spring Boot application leveraging OAuth2 is to add a dependency on the dedicated Spring Boot starter that includes Spring Security and OAuth2 support for Resource Servers.

In the Catalog Service project (catalog-service), open the build.gradle file, and add the new dependency. Remember to refresh or reimport the Gradle dependencies after the new addition.